অবজেক্টের জীবন ও ইনহেরিট্যান্স

কনস্ট্রাক্টর · কপি · ডিস্ট্রাক্টর · অ্যারে · পয়েন্টার · ইনহেরিট্যান্স: ১২ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো লেসন এক পাতায়: লাল = বেশি আসে, সবুজ = যে নিয়মের ওপর বাকি সব দাঁড়িয়ে

➜ সিলেবাসের যে অংশ

OOP সিলেবাস এগিয়ে চলে: "*Constructors, destructors and copy constructors; Array of objects, object pointers, and object references; Inheritance: single and multiple inheritance*"। ক্লাস, অ্যাক্সেস স্পেসিফায়ার ও `static` আগের লেসনে; ভার্চুয়াল ফাংশন, ওভাররাইডিং ও অ্যাবস্ট্রাক্ট ক্লাস পরের লেসনে।

হাই-ইল্ড তালিকা

① কনস্ট্রাক্টর: ক্লাসের নাম, রিটার্ন টাইপ **নেই**
② কোনো *ctor* না লিখলেই ফ্রি ডিফল্ট *ctor*
③ কপি *ctor*: `A(const A&)`
④ ডিফল্ট কপি = শ্যালো
⑤ `~A()`: আর্গুমেন্ট নেই, ওভারলোড নেই
⑥ গঠন বেস → ডিরাইভড; ধ্বংস উল্টো
⑦ *Java* `new A[3]` → ৩টি *null*
⑧ `new` *ctor* ডাকে; `malloc` ডাকে না
⑨ *Java*: ১টি *extends*, অনেক *implements*
⑩ ডায়মন্ড → ভার্চুয়াল বেস ক্লাস

সংকেত হলুদ = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

➜ সূচিপত্র

✓ কনস্ট্রাক্টর: নিয়ম ও প্রকার ... পৃ. ২
✓ কপি কনস্ট্রাক্টর; শ্যালো বনাম ডিপ ... পৃ. ৩
✓ ইনিশিয়ালাইজার লিস্ট ও চেইনিং ... পৃ. ৪
✓ ডিস্ট্রাক্টর ও অবজেক্টের আয়ু ... পৃ. ৫
✓ অবজেক্টের অ্যারে ... পৃ. ৬
✓ পয়েন্টার, `this`, *new/delete* ... পৃ. ৭
✓ ইনহেরিট্যান্স: *is-a* ও পাঁচ প্রকার ... পৃ. ৮
✓ অ্যাক্সেস মোড ও গঠনের ক্রম ... পৃ. ৯
✓ ডায়মন্ড সমস্যা; *Java* ইন্টারফেস ... পৃ. ১০
✓ রাতের পরীক্ষার পাতা ... পৃ. ১১
✓ অনুশীলনী *MCQ* + লিখিত ... পৃ. ১২

১ কনস্ট্রাক্টর: নিয়ম ও প্রকার

সবচেয়ে বেশি আসে ★

সংজ্ঞা কনস্ট্রাক্টর (*Constructor*) একটি বিশেষ মেম্বার ফাংশন, যা অবজেক্ট তৈরির সময় **স্বয়ংক্রিয়ভাবে** চলে এবং অবজেক্টকে একটি বৈধ প্রাথমিক অবস্থায় আনে। এর নাম **ক্লাসের নামের সমান** এবং **কোনো রিটার্ন টাইপ নেই — void-ও নয়**। এটি ওভারলোড করা যায়, `private` হতে পারে (সিঙ্গেলটন), কিন্তু *C++*-এ `virtual` বা `static` হতে পারে না, *Java*-তে `final`, `static` বা `abstract` হতে পারে না। এটি **ইনহেরিট হয় না**।



কনস্ট্রাক্টর = অবজেক্টের জীবনের প্রথম কল; ডিস্ট্রাক্টর = শেষ

প্রতিটি অবজেক্টের জীবন পাঁচ ধাপে

```
class Student {          // C++
    string name;  int age;
public:
    Student() {          // default
        name = "Unknown"; age = 0;
    }
    Student(string n, int a) {  // param.
        name = n; age = a;
    }
    Student(const Student& s) { // copy
        name = s.name; age = s.age;
    }
};
Student s1;              // default
Student s2("Alice", 20); // parameterized
Student s3 = s2;         // copy
```

প্রকার	সিগনেচার	কখন চলে
ডিফল্ট	`A()`	`A a;`
প্যারামিটারাইজড	`A(int x)`	`A a(5);`
কপি	`A(const A&)`	`A b = a;`
ইমপ্লিসিট	কম্পাইলারের	একটিও **না লিখলে**
ওভারলোডেড	ভিন্ন প্যারামিটার	কম্পাইল টাইমে বাছাই

নিয়ম	*C++*	*Java*
ফ্রি ডিফল্ট *ctor*	কেবল **কোনো** *ctor* ঘোষণা না থাকলে	কেবল একটিও না লিখলে; `super()` ডাকে
কপি কনস্ট্রাক্টর	ইমপ্লিসিট, মেম্বার-ভিত্তিক (শ্যালো)	**নেই**: নিজে লেখো বা `clone()`
রিটার্ন টাইপ দিলে	কম্পাইল এরর	`void A(){}` = সাধারণ **মেথড**
`new` ছাড়া অবজেক্ট	`A a;` অবজেক্ট বানায় (স্ট্যাক)	`A a;` কেবল *null* রেফারেন্স

মনে রাখার কৌশল
"**কনস্ট্রাক্টর হলো জন্মসনদ**: তাতে শিশুর নাম থাকে, জন্মের সময় একবারই দেওয়া হয়, আর কিছু ফেরত দেয় না।"

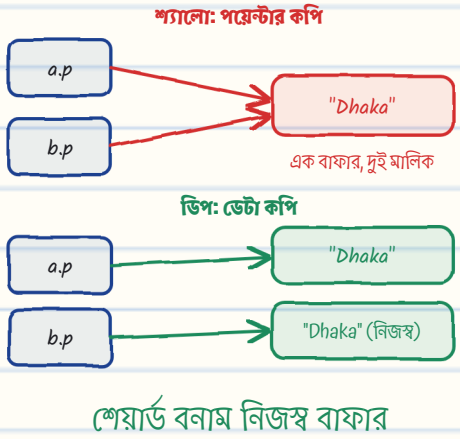
পরীক্ষার ফাঁদ কেবল `A(int)` লিখলে `A a;` / `new A()` **আর কম্পাইল হয় না**। *C++*-এ `A a();` একটি **ফাংশন** ঘোষণা, অবজেক্ট নয়।

সমাধানকৃত উদাহরণ ✍️
কোনটি চলে?
`A a; A b(3); A c = b; A d = 7;` → ডিফল্ট, প্যারামিটারাইজড, **কপি** প্যারামিটারাইজড (ইমপ্লিসিট রূপান্তর; `explicit` দিলে বন্ধ)।

২ কপি কনস্ট্রাক্টর; শ্যালো বনাম ডিপ কপি

লিখিত প্রশ্নের প্রিয় ★

সংজ্ঞা কপি কনস্ট্রাক্টর *(Copy constructor)* একই ক্লাসের বিদ্যমান অবজেক্টের কপি হিসেবে একটি **নতুন** অবজেক্ট তৈরি করে। C++ রূপ: `ClassName(const ClassName& other)`। এটি চলে যখন একটি অবজেক্টকে আরেকটি থেকে **ইনিশিয়ালাইজ** করা হয় (`A b = a;`, `A b(a);`), যখন অবজেক্ট **পাস বাই ভ্যালু** হয়, এবং যখন **রিটার্ন বাই ভ্যালু** হয় (প্রায়ই বাদ পড়ে)। কম্পাইলারের সংস্করণ মেম্বার ধরে ধরে কপি করে — শ্যালো কপি *(Shallow copy)*।



শেয়ার্ড বনাম নিজস্ব বাফার

```
class Str {
    char* p;
public:
    Str(const char* s) {
        p = new char[strlen(s) + 1];
        strcpy(p, s);
    }
    Str(const Str& o) {      // deep copy
        p = new char[strlen(o.p) + 1];
        strcpy(p, o.p);
    }
    ~Str() { delete[] p; }
};
Str a("Dhaka");
Str b = a;   // copy ctor: own buffer
```

বিষয়	শ্যালো কপি	ডিপ কপি *(Deep copy)*
কী কপি হয়	পয়েন্টার (ঠিকানা)	পয়েন্টার যে ডেটা দেখায় সেটি
কে বানায়	কম্পাইলারের কপি *ctor* / `Object.clone()`	নিজের কপি *ctor* / ওভাররাইড করা `clone()`
কপির পরে	*b*-তে বদল *a*-তেও দেখা যায়	অবজেক্ট দুটি স্বাধীন
বিপদ	**ডাবল *delete***, ড্যাংলিং পয়েন্টার	ধীর, বেশি মেমোরি
ঠিক আছে যখন	পয়েন্টার/হ্যান্ডল মেম্বার নেই	ক্লাস হিপ মেমোরি, ফাইল, সকেটের মালিক

কেন `const A&`? **রেফারেন্স**: বাই-ভ্যালু প্যারামিটার হলে কপি বানাতে আরেকটি কপি লাগত → অসীম পুনরাবৃত্তি, তাই C++ `A(A a)` নাকচ করে। ***const***: `const` অবজেক্ট ও টেম্পোরারি কপি করা যায়, আর উৎস না বদলানোর প্রতিশ্রুতি। রুল অব থ্রি *(Rule of three)*: যে ক্লাসের নিজস্ব ডিস্ট্রাক্টর লাগে, তার নিজস্ব কপি কনস্ট্রাক্টর ও কপি অ্যাসাইনমেন্টও লাগে।

Java-তে ডিফল্ট কপি কনস্ট্রাক্টর নেই নিজে লেখো: `Student(Student s) { name = s.name; age = s.age; }`, অথবা `clone()` ওভাররাইড করো। `Object.clone()` **শ্যালো**, `protected`, আর ক্লাস `implements Cloneable` না করলে `CloneNotSupportedException` ছোড়ে।

মনে রাখার কৌশল
"**শ্যালো ঠিকানা কপি করে; ডিপ পুরো বাড়ি।**" এক চাবির দুই মালিক আসবাব নিয়ে ঝগড়া করে।

পরীক্ষার ফাঁদ *b* আগে থেকেই থাকলে `b = a;` ডাকে **অ্যাসাইনমেন্ট অপারেটর**, কপি কনস্ট্রাক্টর নয়। `A b = a;` ইনিশিয়ালাইজেশন → কপি কনস্ট্রাক্টর।

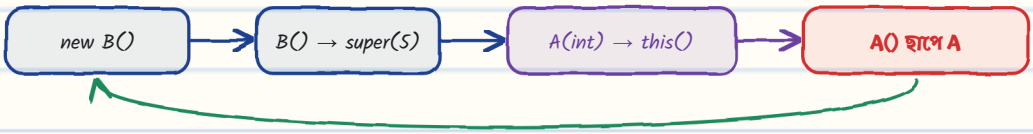
সমাধানকৃত উদাহরণ ✍
কপি গোনো
`void f(A x){}; A a; A b = a; A c(a); f(a); b = c;` → কপি *ctor*: **৩ বার**; শেষটি অ্যাসাইনমেন্ট।

৩ ইনিশিয়ালাইজার লিস্ট ও কনস্ট্রাক্টর চেইনিং আউটপুট প্রশ্ন ★

সংজ্ঞা C++-এর মেম্বার ইনিশিয়ালাইজার লিস্ট *(Member initializer list)* `(A(int v) : x(v), Base(v) {})` বডি চলার **আগেই** বেস ও মেম্বার ইনিশিয়ালাইজ করে। `const` মেম্বার, রেফারেন্স মেম্বার, ডিফল্ট কনস্ট্রাক্টরহীন ক্লাসের মেম্বার এবং আর্গুমেন্ট-চাওয়া বেস ক্লাসের জন্য এটি **বাধ্যতামূলক**। কনস্ট্রাক্টর চেইনিং *(Constructor chaining)* মানে এক কনস্ট্রাক্টর থেকে আরেকটি ডাকা: Java `this(…)` (একই ক্লাস) ও `super(…)` (প্যারেন্ট), C++ `: Base(…)` এবং C++11 থেকে ডেলিগেটিং `A() : A(0) {}`।

```cpp
class Point {                 // C++
    const int x;  int& r;
public:
    Point(int a, int& b)
        : x(a), r(b) {}  // must
};
class P3 : public Point {
    int z;
public:
    P3(int a, int& b, int c)
        : Point(a, b), z(c) {}
    P3(int& b) : P3(0, b, 0) {}
};          // C++11 delegating ctor
```

```java
class A {                     // Java
    A() { System.out.print("A"); }
    A(int x) {
        this();          // 1st line
        System.out.print(x);
    }
}
class B extends A {
    B() {
        super(5);        // 1st line
        System.out.print("B");
    }
}
new B();   // prints A5B
```



ফেরার পথে: A(int) ছাপে 5, তারপর B() ছাপে B → A5B

কল আগে চেইন বেয়ে ওপরে যায়; বডি ছাপে ফেরার পথে

কাজ	C++	Java
প্যারেন্ট *ctor* ডাকা	লিস্টে `: Base(args)`	`super(args)`, **প্রথম লাইন**
নিজের অন্য *ctor*	`: A(args)` *(C++11)*	`this(args)`, প্রথম লাইন
কোনোটিই না লিখলে	বেসের ডিফল্ট *ctor* চলে	কম্পাইলার বসায় `super();`
দুটো একসঙ্গে?	ডেলিগেটিং হলে আর কিছু নয়	**না**, একটিই
ইনিশিয়ালাইজেশনের ক্রম	আগে বেস, তারপর মেম্বার **ঘোষণার ক্রমে**, লিস্টের ক্রমে নয়	

মনে রাখার কৌশল
"**সন্তানের আগে বাবা-মা গড়া হয়**": `super` আগে, বডি শেষে। কল ওঠে ওপরে, প্রিন্ট নামে নিচে।

পরীক্ষার ফাঁদ প্যারেন্টে কেবল `A(int);` `class B extends A { B(){} }` → **কম্পাইল এরর**: লুকানো `super()` কোনো `A()` পায় না। সমাধান: `super(0);`।

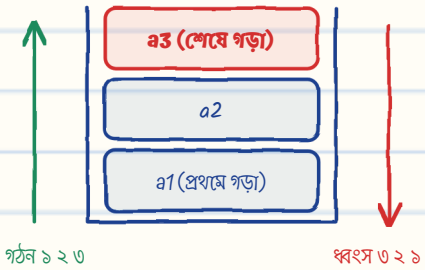
সমাধানকৃত উদাহরণ ✍️
লিস্টের ক্রম
`int a, b;` তারপর `T() : b(1), a(b) {}` → *a* **আগে** সেট হয়, অনির্ধারিত *b* থেকে: **গার্বেজ**।

৪ ডিস্ট্রাক্টর ও অবজেক্টের আয়ু

MCQ + আউটপুট ★

সংজ্ঞা ডিস্ট্রাক্টর (*Destructor*) (`~ClassName()`) অবজেক্টের আয়ু শেষ হলে স্বয়ংক্রিয়ভাবে চলে এবং তার দখলে থাকা জিনিস — হিপ মেমোরি, ফাইল, সকেট, লক — ছেড়ে দেয়। নিয়ম: টিল্ডা + ক্লাসের নাম; **প্যারামিটার নেই, রিটার্ন টাইপ নেই**; তাই **ওভারলোড করা যায় না** — প্রতি ক্লাসে ঠিক একটি; ইনহেরিট হয় না; `virtual` হতে পারে (কনস্ট্রাক্টর কখনো নয়)। অবজেক্ট ধ্বংস হয় গঠনের উল্টো ক্রমে।

অবজেক্টের ধরন	ডিস্ট্রাক্টর কখন চলে
লোকাল (অটোমেটিক)	স্কোপের শেষ }-এ, উল্টো ক্রমে
গ্লোবাল / `static`	প্রোগ্রাম শেষে (`main` ফেরার পর)
হিপ (`new`)	কেবল `delete`-এ — ভুললে **মেমোরি লিক**
টেম্পোরারি	পূর্ণ এক্সপ্রেশনের শেষে
মেম্বার অবজেক্ট	মালিকের ডিস্ট্রাক্টর বডির পরে
অ্যারের উপাদান	সবচেয়ে বড় ইনডেক্স আগে



লোকাল অবজেক্ট স্ট্যাকে: *LIFO*

বিষয়	কনস্ট্রাক্টর	ডিস্ট্রাক্টর
নাম	`A()`	`~A()`
প্যারামিটার / ওভারলোডিং	যেকোনো / হ্যাঁ	**নেই / না**
প্রতি ক্লাসে কয়টি	অনেক	**একটি**
`virtual` হতে পারে	না	হ্যাঁ — পলিমরফিক বেসে দরকার
ইনহেরিট্যান্সে ক্রম	বেস → ডিরাইভড	ডিরাইভড → বেস

ভার্চুয়াল ডিস্ট্রাক্টর (C++) `Base* p = new Derived; delete p;` — `~Base()` `virtual` না হলে কেবল `~Base` চলে (আনডিফাইন্ড বিহেভিয়ার; *Derived* অংশ লিক হয়)। বেস হিসেবে ব্যবহৃত যেকোনো ক্লাসে `virtual ~Base()` দাও। ভার্চুয়াল ডিসপ্যাচ কীভাবে কাজ করে: পরের লেসন।

Java: ডিস্ট্রাক্টর নেই গার্বেজ কালেক্টর (*Garbage collector*) অনাগম্য অবজেক্ট মুক্ত করে, কখন — আগে বলা যায় না; `System.gc()` কেবল অনুরোধ। `finalize()` সর্বোচ্চ একবার চলত, না-ও চলতে পারত, আর এটি **ডেপ্রিকেটেড** (*Java 9; Java 18 থেকে অপসারণের জন্য*)। ফাইল বন্ধ করো *try-with-resources* দিয়ে (`AutoCloseable`)।

মনে রাখার কৌশল "**প্লেটের স্তূপ**: সবশেষে রাখা প্লেটটিই আগে তোলা হয়।" টিল্ডা `~` = বিদায়ের ঢেউ।

পরীক্ষার ফাঁদ গার্বেজ কালেকশন ডিস্ট্রাক্টর **নয়**: এটি কেবল মেমোরি ফেরত নেয়, নির্দিষ্ট সময়ে নয়। `finalize()` চলবেই — এমন নিশ্চয়তা নেই।

সমাধানকৃত উদাহরণ ✍️

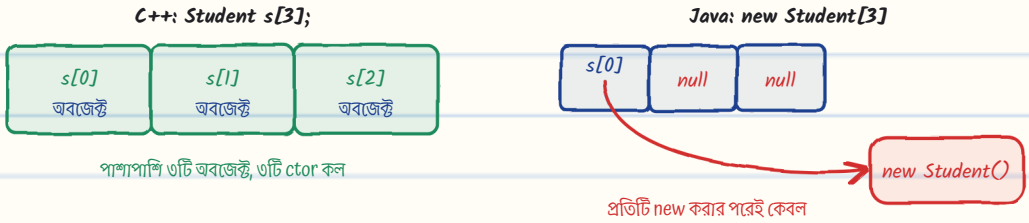
ট্রেস

`{ A a(1); A b(2); A* p = new A(3); }` → গঠন ১ ২ ৩; }-এ: **~2 ~1**

অবজেক্ট ৩ কখনো ধ্বংস হয় না (লিক)।

৫ অবজেক্টের অ্যারে C++ বনাম Java ফাঁদ ★

সংজ্ঞা অবজেক্টের অ্যারে (*Array of objects*) এমন অ্যারে যার প্রতিটি উপাদান একটি ক্লাসের অবজেক্ট। C++-এ `Student s[3];` **তিনটি অবজেক্ট** বানায় এবং ডিফল্ট কনস্ট্রাক্টর তিনবার ডাকে, তাই ক্লাসে অ্যাক্সেসযোগ্য ডিফল্ট কনস্ট্রাক্টর লাগে (নয়তো প্রতিটি উপাদান ইনিশিয়ালাইজ করতে হয়)। **Java**-তে `new Student[3]` একটি অ্যারে বানায় যাতে **তিনটি null রেফারেন্স** — কোনো *Student* অবজেক্ট নেই, কনস্ট্রাক্টর কল নেই — যতক্ষণ না প্রতিটিকে `new` করা হয়।



বাঁয়ে: আসল অবজেক্ট · ডানে: রেফারেন্সের অ্যারে

```cpp
// C++
Student s[3];              // 3 x Student()
Student t[2] = { Student("A", 20),
                 Student("B", 22) };
Student* p = new Student[n];
p[0].display();
delete[] p;    // [] required
```

```java
// Java
Student[] s = new Student[3]; // nulls
s[0].display();     // NullPointerException
for (int i = 0; i < s.length; i++)
    s[i] = new Student();
s[0].name = "Alice";
s[0].display();     // now fine
```

বিষয়	C++ `A arr[3];`	Java `A[] arr = new A[3];`
যা তৈরি হয়	৩টি অবজেক্ট	১টি অ্যারে অবজেক্ট + ৩টি রেফারেন্স
কনস্ট্রাক্টর কল	৩ (ডিফল্ট *ctor*)	0
শুরুর উপাদান	গঠিত অবজেক্ট	`null` (সংখ্যা *0*, *boolean false*)
ক্লাসে ডিফল্ট *ctor* নেই	সব ইনিশিয়ালাইজ না করলে কম্পাইল এরর	সমস্যা নেই
মুক্ত হয়	স্কোপ শেষে, বা `new A[n]`-এর জন্য `delete[]`	গার্বেজ কালেক্টর
দৈর্ঘ্য	সংরক্ষিত নয় (স্থির অ্যারেতে `sizeof` কৌশল)	`arr.length` (ফিল্ড, বন্ধনী নেই)

মনে রাখার কৌশল "**C++ সারি সারি বাড়ি বানায়; Java ছাপে সারি সারি খালি ঠিকানার চিরকুট।**" `new` না করলে বাড়ি নেই।

পরীক্ষার ফাঁদ `new A[n]` মুক্ত করতে হয় `delete[]` দিয়ে; শুধু `delete` আনডিফাইন্ড বিহেভিয়ার। *C++* `A* q[3];` পয়েন্টারের অ্যারে: অবজেক্ট **নেই**।

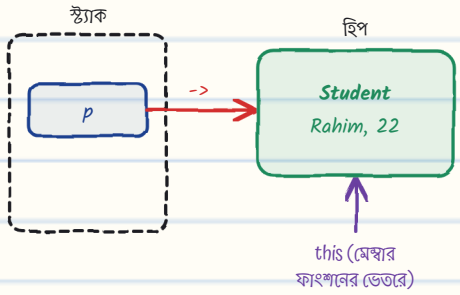
সমাধানকৃত উদাহরণ ✍️

আউটপুট

`A(){cout<<"A";} ~A(){cout<<"~";}` এবং `{ A arr[3]; }` → **AAA~~~**। একই প্রিন্টসহ *Java* `new A[3]`: কিছুই না।

৬ অবজেক্ট পয়েন্টার, this, new/delete, রেফারেন্স MCQতথ্য ★

সংজ্ঞা অবজেক্ট পয়েন্টার (Object pointer) একটি অবজেক্টের ঠিকানা রাখে: `Student* p = &s;`। মেম্বারে পৌঁছাতে অ্যারো: `p->display()` ≡ `(*p).display()`। new হিপে অবজেক্ট বানিয়ে তার ঠিকানা দেয়; delete সেটি ধ্বংস করে। প্রতিটি নন-স্ট্যাটিক মেম্বার ফাংশন একটি লুকানো পয়েন্টার `this` পায়, যা কলকারী অবজেক্টকে দেখায়। Java-তে স্পষ্ট পয়েন্টার নেই: প্রতিটি অবজেক্ট ভ্যারিয়েবল একটি রেফারেন্স (Reference)।



p স্ট্যাকে, অবজেক্ট হিপে

```
class Counter {
    int n = 0;
public:
    Counter& inc() {
        ++this->n;        // this->n == n
        return *this;     // allows chaining
    }
    int get() const { return n; }
};
Counter* c = new Counter;
c->inc().inc().inc();
cout << c->get();  // 3
delete c;  c = nullptr;
```

বিষয়	new / delete (C++)	malloc / free (C)
এটি কী	অপারেটর (ওভারলোড করা যায়)	লাইব্রেরি ফাংশন
কনস্ট্রাক্টর / ডিস্ট্রাক্টর	ডাকে	ডাকে না
আকার ও টাইপ	নিজে হিসাব; ফেরত A*	বাইট হাতে দিতে হয়; ফেরত void*
ব্যর্থ হলে	std::bad_alloc ছোড়ে	NULL ফেরত দেয়
অ্যারে	new A[n] … delete[]	malloc(n*sizeof(A)) … free

বিষয়	C++ পয়েন্টার A* p	C++ রেফারেন্স A& r = a	Java রেফারেন্স
null হতে পারে	হ্যাঁ	না; ইনিশিয়ালাইজ বাধ্যতামূলক	হ্যাঁ (null)
পরে অন্যটিকে দেখানো	হ্যাঁ	না (একটি অ্যালিয়াস)	হ্যাঁ
গাণিতিক / অ্যাক্সেস	p++; -> ও *	নেই; .	নেই; .
অবজেক্ট মুক্ত করা	হাতে delete	মালিকের আয়ু	গার্বেজ কালেক্টর

মনে রাখার কৌশল

"ঠিকানার জন্য অ্যারো, সরাসরি জিনিসের জন্য ডট।"

this = "যে অবজেক্ট কলের জবাব দিল"।

পরীক্ষার ফাঁদ static বা friend ফাংশনে this নেই। this-এ অ্যাসাইন করা যায় না। new-এর সঙ্গে free (বা malloc-এর সঙ্গে delete) মেশানো আনডিফাইন্ড।

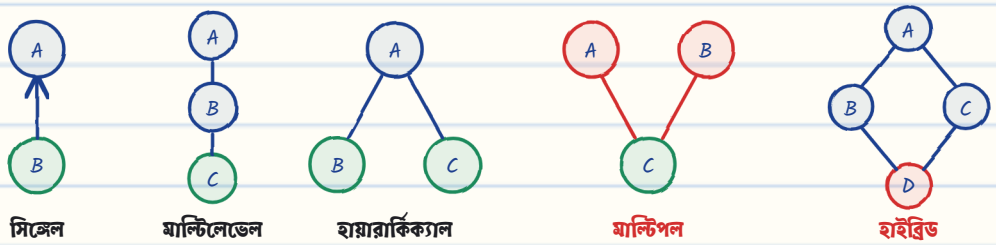
সমাধানকৃত উদাহরণ ✍️

Java অ্যালিয়াস

`Student s = new Student(); s.age = 20; Student r = s; r.age = 21;` → s.age হয় 21। এক অবজেক্ট, দুই রেফারেন্স।

৭ ইনহেরিট্যান্স: is-a ও পাঁচটি প্রকার সর্বোচ্চ হাই-ইল্ড ★

সংজ্ঞা ইনহেরিট্যান্স (*Inheritance*) একটি নতুন **ডিরাইভড** (চাইল্ড, সাব) ক্লাসকে বিদ্যমান **বেস** (প্যারেন্ট, সুপার) ক্লাসের মেম্বার পেতে দেয়, তারপর সে নিজের মেম্বার যোগ বা পুনঃসংজ্ঞায়িত করে। এটি *is-a* সম্পর্ক বোঝায় (*Car is-a Vehicle*), কোড পুনর্ব্যবহার দেয় এবং রান-টাইম পলিমরফিজমের ভিত্তি। সিনট্যাক্স: *C++* `class Car : public Vehicle {…};` · *Java* `class Car extends Vehicle {…}`।



চাইল্ড নিচে; লাল = *Java* ক্লাসে অনুমোদিত নয়

প্রকার	আকৃতি	C++ উদাহরণ	C++ / Java ক্লাস
সিঙ্গেল (*Single*)	*A → B*	`class B : public A`	✓ / ✓ `extends`
মাল্টিলেভেল (*Multilevel*)	চেইন *A → B → C*	`B : A, C : B`	✓ / ✓
হায়ারার্কিক্যাল (*Hierarchical*)	*A → B, A → C*	`B : A, C : A`	✓ / ✓
মাল্টিপল (*Multiple*)	*A, B → C*	`C : public A, public B`	✓ / ✗ (ইন্টারফেসে ✓)
হাইব্রিড (*Hybrid*)	মিশ্রণ, যেমন ডায়মন্ড	*A; B, C : A; D : B, C*	✓ / কেবল ইন্টারফেসে

যা ইনহেরিট হয় না **কনস্ট্রাক্টর** ও **ডিস্ট্রাক্টর** (*C++11*-এ `using Base::Base;` দিয়ে নেওয়া যায়) · কপি **অ্যাসাইনমেন্ট অপারেটর** (ডিরাইভডেরটি লুকিয়ে দেয়) · ***friend*** ফাংশন — বন্ধুত্ব উত্তরাধিকারে যায় না · ***private*** মেম্বার: *C++*-এ ডিরাইভড অবজেক্টের ভেতরে থাকে কিন্তু নাম ধরে ব্যবহার করা যায় না; *Java* বলে এগুলো ইনহেরিট হয় না। `final` ক্লাস (যেমন `String`) এক্সটেন্ড করা যায় না।

বিষয়	*is-a*	*has-a*
কৌশল	ইনহেরিট্যান্স	কম্পোজিশন
উদাহরণ	*Car is-a Vehicle*	*Car has-a Engine*
কাপলিং	টাইট	লুজ
কোড	`extends`	একটি ফিল্ড

মনে রাখার কৌশল পাঁচ প্রকার: "***Some Mice Make Huge Holes***" = *Single, Multilevel, Multiple, Hierarchical, Hybrid*।

পরীক্ষার ফাঁদ

মাল্টিলেভেল ≠ মাল্টিপল। মাল্টিলেভেল = একক ধাপের চেইন (*Java*-তে চলে); মাল্টিপল = একসঙ্গে দুই প্যারেন্ট (*Java* ক্লাসে চলে না)।

সমাধানকৃত উদাহরণ ✍

শ্রেণিবিন্যাস

`C : A, B` → মাল্টিপল। `B : A; C : B` → মাল্টিলেভেল। `B : A; C : A` → হায়ারার্কিক্যাল।

৮ ইনহেরিট্যান্সের অ্যাক্সেস মোড ও গঠনের ক্রম

টেবিল + আউটপুট ★

সংজ্ঞা C++-এ ইনহেরিট্যান্স মোড (*Inheritance mode*) — public, protected বা private — একটি **ছাদ**: ইনহেরিট হওয়া মেম্বার কম দৃশ্যমান হতে পারে, বেশি কখনো নয়। বেসের private মেম্বার ডিরাইভড ক্লাসে **কখনো অ্যাক্সেসযোগ্য নয়**, মোড যা-ই হোক। ডিফল্ট মোড: class → *private*, struct → *public*। *Java*-তে মোড একটিই: extends প্রতিটি অ্যাক্সেস লেভেল যেমন আছে তেমন রাখে।

বেসে ↓	public ইনহেরিট্যান্স	protected ইনহেরিট্যান্স	private ইনহেরিট্যান্স
public	**public**	*protected*	*private*
protected	*protected*	*protected*	*private*
private	**অ্যাক্সেস নেই**	**অ্যাক্সেস নেই**	**অ্যাক্সেস নেই**

```
class X { public:
    X()  { cout << "X"; }
    ~X() { cout << "~X"; } };
class Y { public:
    Y()  { cout << "Y"; }
    ~Y() { cout << "~Y"; } };
class Z : public X {
    Y y;                    // member
public:
    Z()  { cout << "Z"; }
    ~Z() { cout << "~Z"; } };
int main() { Z z; } // XYZ~Z~Y~X
```



ধ্বংস = গঠনকে উল্টো দিক থেকে পড়া

১. আগে **ভার্চুয়াল বেস ক্লাস** (প্রতিটি একবার), তারপর **সরাসরি বেস** বেস-লিস্টের ক্রমে — class C : public B, public A আগে *B*, পরে *A* গড়ে, ইনিশিয়ালাইজার লিস্টে যা-ই লেখা থাক।

২. তারপর **মেম্বার অবজেক্ট** ঘোষণার ক্রমে, শেষে **কনস্ট্রাক্টর বডি**। ধ্বংস হুবহু উল্টো।

৩. **Java:** *static* ব্লক ক্লাস লোডের সময় একবার (প্যারেন্ট আগে); প্রতি new-এ: super() চেইন Object পর্যন্ত, তারপর ফিল্ড ইনিশিয়ালাইজার ও বডি, প্যারেন্ট আগে।

```
class A { int a;            // private
  protected: int b;  public: int c; };
class B : private A {
  void f() { a = 1;   // X base-private
             b = 2;   // ✓ now private in B
             c = 3; }; // ✓ private in B
class C : public B {
  void g() { c = 4; }; // X private in B
B obj;  obj.c = 5;     // X in main
```

সমাধানকৃত উদাহরণ ✍ টেবিল পড়ো B : private A *A*-র *b* ও *c*-কে ***B*-তে private** করে দেয়: *B*-র মেথড এগুলো ব্যবহার করতে পারে; *B*-র চাইল্ড *C* ও main পারে না; *A*-র *a* *B*-র ভেতরেও অনাগম্য। B : public A হলে obj.c ও c = 4 কম্পাইল হয়; *a* **তবুও তালাবদ্ধ**।

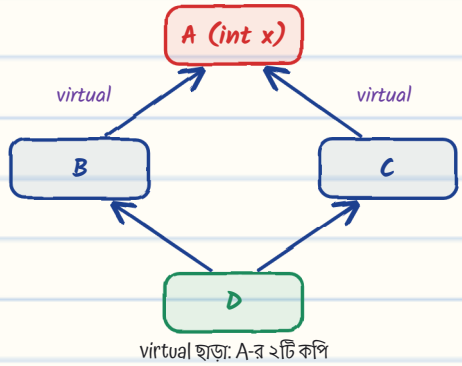
মনে রাখার কৌশল "**মোড হলো ছাদ।**" *public* রেখে দেয়, *protected public*-কে *protected*-এ নামায়, *private* সবকিছুকে *private*-এ নামায়; বেসের *private* তালাবদ্ধই থাকে।

পরীক্ষার ফাঁদ class B : protected A, *A*-তে *public* f(): main-এ B b; b.f(); → **কম্পাইল এরর**; *B*-র নিজের মেথডে f() চলে।

সমাধানকৃত উদাহরণ ✍ আউটপুট A(){cout<<"A";} ~A() {cout<<"a";}, *B*-তে একইভাবে "*B*"/"*b*", class B : public A, B obj; → **ABba**।

৯ ডায়মন্ড সমস্যা ও ভার্চুয়াল বেস ক্লাস প্রায়ই আসে ★

সংজ্ঞা ডায়মন্ড সমস্যা (*Diamond problem*): *B* ও *C* দুটোই *A*-কে ইনহেরিট করে, আর *D* ইনহেরিট করে *B* ও *C*-কে। সাধারণ *D* অবজেক্টে তখন *A*-র **দুটি কপি** থাকে, তাই `d.x` **অস্পষ্ট (ambiguous)** (কম্পাইল এরর)। *C++* সমাধান করে ভার্চুয়াল বেস ক্লাস (*Virtual base class*) দিয়ে: `class B : virtual public A` ও `class C : virtual public A` ভাগ করে **একটি A**। *Java* অবস্থার (*state*) ক্ষেত্রে এটি এড়ায় কেবল একটি প্যারেন্ট ক্লাস দিয়ে।



virtual ছাড়া: A-র ২টি কপি

ডায়মন্ড: এক দাদা-দাদি, দুই পথ

```
class A { public: int x;
    A() { cout << "A"; } };
class B : virtual public A {
public: B() { cout << "B"; } };
class C : virtual public A {
public: C() { cout << "C"; } };
class D : public B, public C {
public: D() { cout << "D"; } };

D d;          // prints ABCD
d.x = 5;      // OK: one shared A
// without virtual: ABACD, and
// d.x is ambiguous (use d.B::x)
```

বিষয়	সাধারণ মাল্টিপল ইনহেরিট্যান্স	virtual বেসসহ
D-র ভেতরে *A*-র কপি	দুটি (একটি *B* দিয়ে, একটি *C* দিয়ে)	**একটি** শেয়ার্ড
`d.x`	অস্পষ্ট → `d.B::x` বা `d.C::x`	ঠিক আছে
A কে গড়ে	*B* ও *C*, প্রত্যেকে নিজের *A*	সবচেয়ে ডিরাইভড ক্লাস *D*, **সবার আগে**
কনস্ট্রাক্টর আউটপুট	*A B A C D*	*A B C D*

```
interface Fly  { default void go()
                 { System.out.print("fly"); } }
interface Swim { default void go()
                 { System.out.print("swim"); } }
class Duck implements Fly, Swim {
    public void go() {        // must override
        Fly.super.go();       // pick one
    }
}
```

Java: ইন্টারফেস দিয়ে একটি `extends`, অনেক `implements`: `class Car extends Vehicle implements Horn, Gps`। *Java 8* থেকে ইন্টারফেসে `default` মেথড থাকতে পারে, তাই মেথড-স্তরের ডায়মন্ড সম্ভব: একই সিগনেচারের দুটি *default* → ক্লাস ওভাররাইড না করা পর্যন্ত **কম্পাইল এরর**; চাইলে `Fly.super.go()` ডাকা যায়। ক্লাসের মেথড ইন্টারফেসের *default*-কে হারায়।

মনে রাখার কৌশল "**ভার্চুয়াল বেস = এক শেয়ার্ড দাদি।**" দুই প্যারেন্ট, কিন্তু পারিবারিক দাওয়াতে দাদি একজনই, দুজন নন।

পরীক্ষার ফাঁদ `virtual` বসে **মাঝের** ক্লাসে (*B* ও *C*), *D*-তে নয়। **ভার্চুয়াল বেস ক্লাস** আর **ভার্চুয়াল ফাংশন** এক নয়; দ্বিতীয়টি পরের লেসনে।

সমাধানকৃত উদাহরণ ✍️

আকারের দৃষ্টিতে

4-বাইট `int x` হলে সাধারণ ডায়মন্ড *D*-তে *x*-এর **২টি** কপি; ভার্চুয়াল সংস্করণে ১টি (সঙ্গে লুকানো বেস পয়েন্টার)।

পরীক্ষার আগের রাতের পাতা

তথ্য পাতা

কনস্ট্রাক্টর: ক্লাসের নাম, রিটার্ন টাইপ নেই
কোনো *ctor* না লিখলেই ফ্রি ডিফল্ট *ctor*
ডিফল্ট কপি = শ্যালো, মেম্বার-ভিত্তিক
গঠন: ভার্চুয়াল বেস → বেস → মেম্বার → বডি
Java `super()/this()`: প্রথম স্টেটমেন্ট
`new` → *ctor* + `bad_alloc`; `malloc` → *NULL*
`p->f()` ≡ `(*p).f()`

ডিস্ট্রাক্টর: `~A()`, আর্গুমেন্ট নেই, ক্লাসে একটি
কপি *ctor*: `A(const A&)`
রুল অব থ্রি: *dtor* + কপি *ctor* + *operator=*
ধ্বংস: হুবহু উল্টো
Java: ডিস্ট্রাক্টর নেই; *GC*; *finalize* ডেপ্রিকেটেড
`new[]`-এর জোড়া `delete[]`
static / *friend* ফাংশনে `this` নেই

→ MCQ-র জন্য সেরা ১২টি এক-লাইনের কথা

1. কনস্ট্রাক্টর ওভারলোড হয়; ডিস্ট্রাক্টর হয় না
2. কনস্ট্রাক্টর *virtual* হয় না; ডিস্ট্রাক্টর হয়
3. কনস্ট্রাক্টর ইনহেরিট হয় না
4. `A b = a;` → কপি *ctor*; `b = a;` → *operator=*
5. *C++* `A arr[3]` → ৩ বার ডিফল্ট *ctor*
6. *Java* `new A[3]` → ০ *ctor* কল, ৩টি *null*
7. ইনহেরিট্যান্স = *is-a*; কম্পোজিশন = *has-a*
8. মাল্টিলেভেল = চেইন; মাল্টিপল = দুই প্যারেন্ট
9. *Java*: মাল্টিপল ইনহেরিট্যান্স কেবল ইন্টারফেসে
10. বেসের *private*: ডিরাইভডে কখনো অ্যাক্সেস নেই
11. *C++* `class` ডিফল্টে *private* ইনহেরিট করে
12. ডায়মন্ড → `virtual` বেস ক্লাস

সেরা ফাঁদ `A a();` একটি ফাংশন · `A(int)` লিখলে ফ্রি ডিফল্ট *ctor* চলে যায় · বাই-ভ্যালু কপি *ctor* অবৈধ · *GC* ≠ ডিস্ট্রাক্টর · `new[]`-এ `delete` = *UB* · ইনিশিয়ালাইজেশন ক্রম = ঘোষণার ক্রম · `virtual` *B* ও *C*-তে, *D*-তে নয়।

সব কৌশল একসঙ্গে জন্মসনদ · শ্যালো ঠিকানা কপি করে, ডিপ বাড়ি · সন্তানের আগে বাবা-মা · প্লেটের স্তূপ · বাড়ি বনাম ঠিকানার চিরকুট · ঠিকানায় অ্যারো, জিনিসে ডট · *Some Mice Make Huge Holes* · মোড হলো ছাদ · এক শেয়ার্ড দাদি।



তিন ধারণা, তিন লাইন

সমাধানকৃত উদাহরণ ✍ এক কিওয়ার্ডে আউটপুট বদলায় ডায়মন্ডে `B : virtual public A, C : virtual public A, D : public B, public C` ছাপে "ABCD"। দুটো `virtual` সরালে একই `D d;` ছাপে *"ABACD"* — আর `d.x` কম্পাইল হয় না।

✓ অনুশীলনী MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

আগে চেষ্টা, পরে মেলাও

১. কনস্ট্রাক্টরের রিটার্ন টাইপ:
ক) *void* খ) *int*
গ) নেই ঘ) ক্লাস টাইপ

২. ক্লাস *A*-র বৈধ *C++* কপি কনস্ট্রাক্টর কোনটি?
ক) `A(A a)` খ) `A(const A& a)`
গ) `A(A* a)` ঘ) `void A(A& a)`

৩. *C++* ডিস্ট্রাক্টর পারে:
ক) ওভারলোড হতে খ) আর্গুমেন্ট নিতে
গ) *int* ফেরত দিতে ঘ) *virtual* হতে

৪. *Java* `Student[] s = new Student[3];` তৈরি করে:
ক) ৩টি *null* রেফারেন্স, *Student* নেই খ) ৩টি *Student* অবজেক্ট
গ) কম্পাইল এরর ঘ) ডিফল্ট মানসহ ৩টি অবজেক্ট

৫. *A ctor/dtor*-এ "A"/"a", *B* একইভাবে "B"/"b" ছাপে; `class B : public A; B obj;` ছাপে:
ক) *ABab* খ) *ABba*
গ) *BAba* ঘ) *BAab*

৬. *C* ইনহেরিট করে *B*-কে, *B* ইনহেরিট করে *A*-কে। এটি:
ক) মাল্টিপল খ) হায়ারার্কিক্যাল
গ) মাল্টিলেভেল ঘ) হাইব্রিড

৭. *C++*-এ ডায়মন্ড সমস্যার সমাধান:
ক) *friend* ফাংশন খ) *virtual* ফাংশন
গ) *static* মেম্বার ঘ) *virtual* বেস ক্লাস

৮. `protected` ইনহেরিট্যান্সে বেসের `public` মেম্বার হয়:
ক) *protected* খ) *public*
গ) *private* ঘ) অ্যাক্সেসহীন

৯. কোনটি `new`-এর ক্ষেত্রে সত্য, `malloc`-এর নয়?
ক) `void*` ফেরত দেয় খ) বাইটে আকার লাগে
গ) কনস্ট্রাক্টর ডাকে ঘ) ব্যর্থ হলে *NULL* দেয়

১০. কোন *Java* কিওয়ার্ড প্যারেন্ট ক্লাসের কনস্ট্রাক্টর ডাকে?
ক) *this* খ) *super*
গ) *extends* ঘ) *parent*

উত্তরমালা → ১-গ · ২-খ · ৩-ঘ · ৪-ক · ৫-খ · ৬-গ · ৭-ঘ · ৮-ক · ৯-গ · ১০-খ

লিখিত প্রশ্ন কপি কনস্ট্রাক্টর কী? এটি কখন ডাকা হয়? হিপ মেমোরির মালিক একটি *C++* ক্লাস দিয়ে শ্যালো ও ডিপ কপি ব্যাখ্যা করো, এবং মাল্টিলেভেল ইনহেরিট্যান্সে কনস্ট্রাক্টর ও ডিস্ট্রাক্টর কলের ক্রম লেখো। ১০ নম্বর

১ **সংজ্ঞা (২)** — `A(const A& o)` কনস্ট্রাক্টর, যা বিদ্যমান অবজেক্ট থেকে নতুন অবজেক্ট গড়ে; রেফারেন্সে (বাই-ভ্যালু হলে অসীম পুনরাবৃত্তি), `const` যাতে *const* অবজেক্ট ও টেম্পোরারি নেওয়া যায়। ২

২ **কখন ডাকা হয় (২)** — `A b = a;` / `A b(a);`, পাস বাই ভ্যালু, রিটার্ন বাই ভ্যালু; `b = a;`-এ নয় (অ্যাসাইনমেন্ট অপারেটর)। ২

৩ **শ্যালো বনাম ডিপ (৪)** — ডিফল্ট কপি পয়েন্টার কপি করে → শেয়ার্ড বাফার, ডাবল *delete*; ডিপ কপি নতুন বাফার নিয়ে ডেটা কপি করে (`new char[]`, `strcpy`, `delete[]`সহ *Str* ক্লাস); রুল অব থ্রি। ৪

৪ **ক্রম (২)** — *A* → *B* → *C* হলে: কনস্ট্রাক্টর *A*, *B*, *C*; ডিস্ট্রাক্টর *~C*, *~B*, *~A*। ২

→ পরীক্ষার আগে ✓

- ✓ কনস্ট্রাক্টর ও ডিস্ট্রাক্টরের নিয়ম বলতে পারি
- ✓ ডিপ-কপি কনস্ট্রাক্টর লিখতে পারি
- ✓ গঠন/ধ্বংসের ক্রম ট্রেস করতে পারি
- ✓ অ্যাক্সেস-মোড টেবিল পূরণ করতে পারি
- ✓ পাঁচ প্রকার ইনহেরিট্যান্স আঁকতে পারি
- ✓ `virtual` দিয়ে ডায়মন্ড ঠিক করতে পারি